CW3 – Part 2 Specification

Car Rental Management System (CRMS) – Implementation

CRMS Interface and Supporting Classes

The CRMS interface outlines the main methods needed for managing the car booking, processing, and finalization phases in a rental system.

First, we need to establish the car rental management system, which comprises **clients** who need to rent cars, **agents** who will process the bookings, and **cars** available for rental at various branches.

The method **InitializeRentalSystem** sets up the rental environment as indicated by the parameters specifying the number of clients, cars, and agents. The returned **RentalSystem** by InitializeRentalSystem will include collections of clients making bookings, available agents to manage contracts, and cars to be allocated.

Three primary methods are designed to operate the car rental lifecycle: **book, process, and finalize**.

- The first phase of the rental journey starts when a client books a car and is assigned an agent for contract handling. This is implemented by the **Book** method.
- The **Book** method should compile a **BookingRecord** that contains all details about the client making the booking, the selected car, and the assigned rental agent. Once the booking record connecting these three parties has been generated, the next phase involves processing the booking.
- The **Process** method takes a booking record and updates it with applied insurance, discounts, and branch-specific policies. The processed record represents a confirmed booking ready for payment.
- Finally, the **Finalize** method takes the processed booking record and completes the transaction, including payment verification and pickup details. The **FinalizedRecord** confirms that the client is ready to collect the car and start their rental.

Types of Records (BookingRecord, ProcessedRecord, FinalizedRecord)

Despite the **FinalizedRecord** being a **ProcessedRecord**, and the **ProcessedRecord** being a **BookingRecord**, there are key differences among the three types of records.

- The **BookingRecord** is like the initial booking form the client completes on the rental system. It includes sections for the client's details, car details, and assigned agent's information. Within these sections, there are subsections such as rental duration, pickup and drop-off locations, and optional extras. At this stage, no processing (insurance, discounts, or branch rules) has been applied yet.
- Once the booking has been reviewed by an agent, the **ProcessedRecord** is generated. This record is considered a **BookingRecord** with added processing information, which includes insurance options applied, discounts offered, and any deposit requirements. The processed record is thus an enriched booking, awaiting payment confirmation.

- The **FinalizedRecord** is the completed rental agreement. It includes final payment confirmation, pickup details, and rental terms. At this stage, the booking is locked in, the client has provided the required payment/deposit, and the car is officially reserved for collection.

Methods Details

- **InitializeRentalSystem(int numOfClients, int numOfCars, int numOfAgents): RentalSystem**
  Sets up the rental environment. It takes in the number of clients, cars, and agents. The method initializes the rental system with collections of clients, available cars, and agents.
- **Book(Client c, Car car, Agent a): BookingRecord**
  Handles the initial booking process. It takes a client, the car being rented, and the agent managing the contract. It returns a booking record with details of all three parties.
- **Process(BookingRecord br): ProcessedRecord**
  Represents the processing phase of the booking. It takes a booking record as input, applies insurance and discounts, and generates a processed record.
- **Finalize(ProcessedRecord pr): FinalizedRecord**
  Finalizes the rental process by confirming payment, deposit, and pickup arrangements. It returns a finalized record, confirming the client's rental agreement.

Tasks and Outcomes: Your team is required to produce the following:

Task 1: Implementation of Supporting Classes

You are required to implement the necessary supporting classes for the **CRMS interface**.

1. RentalSystem Class

- **Client Class**
  - *LicenseDetails Class: Contains information about the client's driving license number, validity period, and any restrictions.*
  - *ContactDetails Class: Includes the client's contact information such as address, phone number, and email.*
- **Car Class**
  - *Features Class: Contains information about the car's fuel type, transmission, seating capacity, and category (e.g., Economy, SUV, Luxury).*
  - *InsuranceDetails Class: Includes insurance provider details, coverage options, and optional add-ons (e.g., premium cover, accident protection).*
- **Agent Class**
  - *Branch Class: Provides the location of the rental branch where the agent works.*
  - *Schedule Class: Manages the agent's working hours and availability for handling bookings.*

2. BookingRecord Class: Stores the initial booking details, including:

- *The client making the booking.*
- *The car being requested.*
- *The agent handling the booking.*
- *Rental start date, end date, and base cost.*

3. ProcessedRecord Class (extends BookingRecord)

- Extends the BookingRecord with additional processing details:
  - *InsuranceOption Class: Specifies insurance applied to the booking (e.g., standard, premium).*
  - *Discount Class: Documents any discounts (e.g., loyalty, promotional, corporate).*

4. FinalizedRecord Class (extends ProcessedRecord)

- Represents the completed and confirmed booking after payment.
  - *PaymentDetails Class: Includes deposit amount, total rental cost, and outstanding balance if any.*
  - *PickupDetails Class: Provides details about the car pickup, including pickup branch, date, and special instructions.*

Task 2: Class Implementing CRMS Interface – CarRentalImplementer

Write a class named **CarRentalImplementer** to implement the above Car Rental Management System (CRMS) interface. This class will handle the **four main methods** required for managing the rental journey through the system:

1. **InitializeRentalSystem**
   - Sets up the rental environment by creating collections of clients, cars, and agents based on the given numbers.
   - Ensures that the system is ready to perform bookings, processing, and finalizations.
2. **Book**
   - Handles the booking of a car by a client.
   - Creates a BookingRecord that connects the client, chosen car, and assigned agent.
   - The BookingRecord includes the basic rental information such as duration and base cost.
3. **Process**
   - Manages the processing of a booking.
   - Takes a BookingRecord, applies insurance, discounts, and rental branch rules.
   - Produces a ProcessedRecord with updated cost and conditions.
4. **Finalize**
   - Finalizes the client's booking.
   - Takes a ProcessedRecord, verifies the payment, and generates a FinalizedRecord.
   - The FinalizedRecord confirms deposit, total cost, pickup date, and branch details.

Task 3: Test Class of CarRentalImplementer – TestRental

Write a test class named **TestRental** to test the CarRentalImplementer class.

Steps:

1. Initialize the rental system with ONE client, TWO agents, and ONE car.
2. Create TWO booking records for the same client and car, each handled by a different agent.
3. Process the two booking records, applying different insurance or discount terms.
4. Compare the total costs of the two processed records.
5. Finalize the booking that offers the better deal to the client.

Task 4: Teamwork Evidence (Described in Part1)

Submission: One .Zip file contains:

- **CW3 Part 2 with solutions of Tasks 1–3.**
- **CW3 Part 2 Roles & Contribution Form.**
- **Teamwork Evidence**

Only **ONE submission per team** is required.

Part 2: Marking Criteria

Part 1	Max. Grade
Task 1: Implementation of Supporting Classes	10 %
Task 2: Class Implementing CRMS Interface	10 %
Task 3: Test Class	10 %
Task 4: Teamwork Evidence	10 %
Part 2 Weight	40%

⚠ **Restrictions:** The use of AI code-generation tools (e.g., ChatGPT, GitHub Copilot, etc.) is strictly prohibited for producing solutions. All submissions will be subject to plagiarism detection and code similarity checks. Any evidence of AI-generated or plagiarized work will result in penalties according to academic integrity regulations.